CURSO UNIVERSITARIO

SW505 - Construcción de Software I

Semana 3 — Anticipación al cambio, estándares de codificación y fundamentos de construcción asistida por IA

M. Sc. Juan Luis Herencia Guerra

CONTENIDO DE LA SESIÓN

Agenda

01

Diseño para el cambio

Puntos de variación y mecanismos de extensión (polimorfismo, DI, configuración, Strategy).

02

Reutilización y estándares

Niveles de reutilización, nomenclatura, formato, comentarios y organización de archivos.

03

Construcción asistida por IA

Prompting como especificación, descomposición de tareas, riesgos y responsabilidad profesional.

04

Documentación y cierre

Documentación técnica mínima y síntesis de la semana.

BLOQUE 1

Diseño para el cambio



Puntos de variación y mecanismos que aíslan el costo del cambio dentro de una frontera controlada.

El cambio como inevitable y anticipable

Todo sistema de software cambia: los requisitos evolucionan, el entorno tecnológico se transforma, el negocio se adapta.

✓ **La pregunta correcta**	No es si el sistema cambiará, sino cuándo y dónde.
✓ **Aislamiento del cambio**	El constructor que anticipa el cambio diseña mecanismos de aislamiento que evitan que el costo de modificar una parte se propague al resto del sistema.
✓ **Punto de variación**	Es una decisión de diseño que se sabe que puede cambiar. Encapsularlo significa que el cambio afecta solo al código dentro de esa frontera.

Cuatro mecanismos de extensión

Formas concretas de encapsular un punto de variación sin que el resto del sistema conozca los detalles.

- ✓ Polimorfismo — el comportamiento variable se expresa como implementaciones de una interfaz común.
- ✓ Inyección de dependencias — los colaboradores de un módulo se suministran desde fuera, en lugar de instanciarlos internamente.
- ✓ Parámetros de configuración — los valores que cambian entre entornos se externalizan, no se incrustan en el código.
- ✓ Patrón Strategy — el algoritmo variable se encapsula en un objeto intercambiable, independiente del contexto que lo invoca.

Polimorfismo

El comportamiento variable se expresa como implementaciones de una interfaz común.

- ✓ El código cliente (pagar) no sabe qué implementación usa.
- ✓ Agregar un nuevo método de pago no requiere modificar el código existente.
- ✓ Reduce el acoplamiento entre el flujo de negocio y los detalles de cada método.

JAVA

```
interface MetodoPago {
  void procesar(double monto);
}

class PagoTarjeta implements MetodoPago {
  public void procesar(double monto) {
    // lógica específica de tarjeta
  }
}

class PagoYape implements MetodoPago {
  public void procesar(double monto) {
    // lógica específica de Yape
  }
}

void pagar(MetodoPago metodo, double monto) {
  metodo.procesar(monto);
}
```

El cliente pagar(...) permanece estable aunque se agreguen nuevos métodos de pago.

Inyección de dependencias

Los colaboradores de un módulo se suministran desde fuera, en lugar de instanciarlos internamente.

- ✓ Facilita el reemplazo del colaborador sin tocar la clase que lo usa.
- ✓ Facilita la prueba: se puede inyectar un doble de prueba en lugar del servicio real.
- ✓ El objeto Pedido no decide con qué notificador trabaja.

PYTHON

```python
class ServicioNotificacion:
    def enviar(self, mensaje: str) -> None:
        ...

class Pedido:
    def __init__(self, notificador: ServicioNotificacion):
        self._notificador = notificador

    def confirmar(self):
        self._notificador.enviar(
            "Pedido confirmado"
        )

# se inyecta desde fuera
pedido = Pedido(EmailNotificador())
```

Cambiar de EmailNotificador a SmsNotificador no requiere tocar la clase Pedido.

Parámetros de configuración

Los valores que cambian entre entornos —URLs, umbrales, nombres de archivo— se externalizan a archivos de configuración; no se incrustan en el código.

✓ **Por qué importa**	Cambiar de entorno (desarrollo, pruebas, producción) no debería requerir recompilar ni modificar el código fuente.
✓ **Dónde vive el valor**	Archivos .env, variables de entorno, servicios de configuración centralizados (por ejemplo, un gestor de secretos).
✓ **Riesgo si se ignora**	Valores incrustados en el código («hardcodeados») obligan a un despliegue nuevo por cada cambio de umbral o URL, y facilitan filtrar credenciales en el repositorio.

Patrón Strategy

El algoritmo variable se encapsula en un objeto intercambiable, independiente del contexto que lo invoca.

- ✓ Carrito no implementa la lógica de descuento; la delega.
- ✓ Se pueden agregar nuevas estrategias sin modificar Carrito.
- ✓ Distinto del polimorfismo simple: aquí el objeto completo representa un algoritmo, no una variante de comportamiento de una entidad de negocio.

JAVA

```
interface EstrategiaDescuento {
  double aplicar(double precio);
}

class DescuentoNavidad
    implements EstrategiaDescuento {
  public double aplicar(double precio) {
    return precio * 0.85;
  }
}

class Carrito {
  private EstrategiaDescuento estrategia;

  double total(double precio) {
    return estrategia.aplicar(precio);
  }
}
```

La estrategia se inyecta en Carrito (a menudo combinando Strategy con inyección de dependencias).

Comparativa de los cuatro mecanismos

Mecanismo	Qué encapsula	Cuándo usarlo
Polimorfismo	Variantes de un mismo comportamiento de negocio	Múltiples implementaciones de una misma operación (pagos, formatos de salida)
Inyección de dependencias	El colaborador concreto que usa un módulo	Siempre que un módulo dependa de un servicio externo o reemplazable
Parámetros de configuración	Valores que cambian según el entorno	URLs, umbrales, credenciales, nombres de archivo
Patrón Strategy	Un algoritmo completo e intercambiable	Reglas de negocio que cambian con frecuencia (descuentos, tarifas, validaciones)

BLOQUE 2

Reutilización y estándares de codificación



De copiar y pegar a librerías versionadas; de la convención tácita al estándar acordado por el equipo.

Niveles de reutilización de componentes

La reutilización reduce defectos porque el código reutilizado ya fue probado en contextos anteriores.

✓ **Copiar y pegar**	El nivel más bajo y más riesgoso: cada copia debe actualizarse por separado cuando la lógica cambia.
✓ **Patrones de diseño**	Soluciones reutilizables a nivel de estructura, no de código — como Strategy, visto en el bloque anterior.
✓ **Librerías versionadas**	El nivel más alto: componentes empaquetados, con control de versiones explícito y ciclo de pruebas propio.

Qué es un estándar de codificación

Un conjunto de reglas acordadas por el equipo que determinan cómo debe escribirse el código.

✓ **Propósito**	No es estético: reduce la carga cognitiva al leer código escrito por otros —incluido código generado por una IA— y elimina ambigüedades que puedan inducir errores.
✓ **Alcance**	Cubre nomenclatura, formato, comentarios y organización de archivos — los cuatro ejes que se detallan a continuación.
✓ **Quién lo aplica**	Todo el equipo, de forma consistente; hoy gran parte se automatiza mediante linters y formateadores.

Nomenclatura significativa

Los nombres deben revelar la intención.

- Evitar abreviaturas crípticas y nombres de una sola letra (salvo en bucles triviales).
- Los nombres de clase deben ser sustantivos; los de función, verbos o frases verbales.
- Usar nombres pronunciables: facilitan la comunicación oral en el equipo.
- Usar nombres buscables: los números mágicos y los nombres de una letra no se pueden buscar en el código.

Convenciones de formato y automatización

Indentación, longitud de línea, posición de llaves: la consistencia reduce el ruido visual al leer.

Antes	Estas reglas se discutían manualmente en cada revisión de código, generando fricción y decisiones inconsistentes entre proyectos.
Hoy	Se delegan a formateadores automáticos que se ejecutan en cada guardado o en el pipeline: Prettier (JS/TS), Black (Python), gofmt (Go).
Beneficio	Elimina la discusión de estilo en la revisión de código, dejando esa energía para discutir corrección y diseño.

Comentarios: el porqué, no el qué

EVITAR

```
// incrementa i en 1
i++;

// llama a la función
procesarLote(datos);
```

PREFERIR

```
// avanzamos al siguiente lote:
// la API pagina de a 50 registros
i++;

// se reintenta 3 veces porque el
// proveedor externo es inestable
procesarLote(datos);
```

El código debe ser autoexplicativo. Un comentario que reitera el código es ruido; uno que explica una decisión no obvia es información.

Organización de archivos

Dónde viven las clases, cómo se nombran los paquetes o módulos, cómo se estructura el proyecto.

✓ **Convención sobre configuración**	Estructuras predecibles (por ejemplo, separar src/, test/, config/) permiten que cualquier persona del equipo encuentre un archivo sin preguntar.
✓ **Nombres de paquetes y módulos**	Deben reflejar el dominio del problema, no detalles de implementación.
✓ **Coherencia con el estándar del equipo**	La organización de archivos es tan parte del estándar de codificación como la nomenclatura o el formato.

BLOQUE 3

Construcción asistida por IA



El código sugerido por un asistente de IA está sujeto a los mismos estándares de calidad que el código escrito a mano.

Por qué este tema entra al temario

2026

Hacia Adelante

El código producido con asistencia de modelos de lenguaje (LLM) ya representa una fracción significativa del código escrito en la industria.

- ✓ Tratarlo como un tema aparte sería ignorar cómo se construye software hoy.
- ✓ Este temario lo introduce como una habilidad de construcción más, no como un atajo.
- ✓ Está sujeto a los mismos estándares de calidad que el código escrito a mano: legibilidad, nomenclatura, pruebas y revisión.

Prompting como especificación

Pedirle a un asistente que genere código exige describir con precisión el contrato de la función.

✓ **Entradas y salidas**	Qué recibe la función y qué debe retornar, con sus tipos.
✓ **Casos borde**	Qué debe ocurrir con datos vacíos, nulos o fuera de rango.
✓ **Restricciones**	Lenguaje, librerías permitidas, estilo del equipo, requisitos de seguridad.
✓ **En esencia**	Es escribir una especificación antes de escribir la implementación — la misma disciplina de siempre, aplicada a un nuevo colaborador.

Un prompt es una especificación: calidad de entrada, calidad de salida

MAL ESPECIFICADO

"Hazme una función para validar un email."

No dice el lenguaje, el tipo de retorno, qué formato exacto se acepta, ni cómo probarlo. El asistente debe adivinar — y adivinará distinto cada vez.

BIEN ESPECIFICADO

"En Python, escribe validar_email(texto: str) -> bool que retorne True solo si el texto tiene el formato usuario@dominio.tld, sin librerías externas. Agrega 3 pruebas con pytest: email válido, sin arroba, y dominio vacío."

Denny et al. (2024): la descomposición y la especificación precisa son las estrategias más citadas por desarrolladores para obtener resultados útiles de un LLM.
https://juholeinonen.com/assets/pdf/denny2024computing.pdf

Descomponer la tarea, no pedir la solución completa

Dividir un problema grande en solicitudes pequeñas y verificables produce mejores resultados que pedir una solución completa de una sola vez (Denny et al., 2024).

01

Definir el contrato

Especificar entradas, salidas y casos borde de la unidad más pequeña posible.

02

Solicitar una porción

Pedir al asistente solo esa unidad, no el sistema completo.

03

Verificar antes de avanzar

Leer, entender y probar el resultado antes de pedir la siguiente pieza.

04

Integrar y repetir

Combinar las piezas verificadas y continuar con la siguiente unidad.

El asistente como colaborador junior, no como autoridad

El código generado debe leerse, entenderse y probarse con el mismo rigor que el código de un compañero de equipo.

✓ **Nunca se integra sin revisión humana**	El ciclo estándar de la semana 6* para la construcción (planificar, ejecutar y entregar un resultado completo) aplica sin excepción: entender cada línea, tener una prueba que la respalde, y seguir las convenciones del proyecto. *Enfoque de gestión como el marco Shape Up a cambio del tradicional sprint de SCRUM de 1 a 4 semanas.
✓ **Analogía útil**	Igual que un colaborador junior con mucho conocimiento pero sin contexto del proyecto: rápido, prolífico, pero no infalible.
✓ **El criterio sigue siendo humano**	El asistente propone; la persona decide si esa propuesta es correcta para este sistema, en este momento.

Riesgos de la aceptación acrítica (“vibe coding”*)

Aceptar sugerencias de código sin comprenderlas produce sistemas que compilan y pasan pruebas superficiales, pero esconden problemas serios.

✓ **Deuda técnica invisible**	Nadie en el equipo puede explicar por qué el código funciona como funciona.
✓ **Vulnerabilidades no detectadas**	Un prompt sin restricciones de seguridad puede reproducir patrones inseguros conocidos.
✓ **Falsa sensación de calidad**	“Compila y pasa las pruebas” no es lo mismo que “es correcto, legible y listo para el cambio”

(*) El término fue popularizado a principios de 2025 por Andrej Karpathy (cofundador de OpenAI). Significa "programar por vibraciones": en lugar de teclear cada línea de un lenguaje como JavaScript o Python, mantienes una charla en lenguaje natural con un asistente de IA. Vibe coding: programming through conversation with artificial intelligence.

Aceptado sin revisión: funciona… y es vulnerable

Código sugerido por un asistente sin restricciones de seguridad en el prompt.

- ✓ Compila y pasa la prueba del camino feliz.
- ✓ Concatena directamente la entrada del usuario dentro de la consulta SQL.
- ✓ Es vulnerable a inyección SQL: una entrada maliciosa puede alterar la consulta completa.

PYTHON

```python
def buscar_usuario(nombre):
    query = (
        "SELECT * FROM usuarios "
        "WHERE nombre = '" + nombre + "'"
    )
    return db.execute(query)

# nombre = "x' OR '1'='1"
# altera el significado de la
# consulta por completo
```

Ninguna prueba "feliz" detecta este defecto: hay que saber buscarlo.

Corregido tras la revisión disciplinada

Misma funcionalidad, revisada con el estándar de la semana 6* antes de integrarse. (*Shape Up - Enfoque de construcción de software)

- ✓ Usa una consulta parametrizada en lugar de concatenar texto.
- ✓ El motor de base de datos trata la entrada como dato, nunca como código.
- ✓ La persona que integra el código sigue siendo responsable de su seguridad, la haya escrito ella o un asistente.

PYTHON

```
def buscar_usuario(nombre):
    query = (
        "SELECT * FROM usuarios "
        "WHERE nombre = %s"
    )
    return db.execute(query, (nombre,))

# la entrada se trata como dato,
# nunca como parte de la consulta
```

Shape Up opera en un ritmo constante que combina el desarrollo con el descanso y la estrategia:

Fase	Duración	¿Qué se hace?
Ciclo de Construcción (*Building Cycle*)	**6 semanas**	Los equipos se enfocan exclusivamente en construir y entregar los proyectos asignados. Tienen prohibido ser interrumpidos con nuevas ideas o soporte técnico menor.
Periodo de Enfriamiento (*Cool-down*)	**2 semanas**	Un espacio libre de Sprints. El equipo lo usa para corregir errores menores, investigar nuevas tecnologías, proponer mejoras por iniciativa propia o descansar.

Misma solicitud, mismo asistente — la diferencia la hizo la revisión humana.

Responsabilidad profesional

Quien integra código generado por IA a un repositorio es su autor a todos los efectos.

✓ **De calidad**	El código integrado responde a los mismos estándares de legibilidad y diseño que el resto del sistema.
✓ **De mantenimiento**	Quien lo integró debe poder explicarlo y modificarlo meses después, sin depender de volver a preguntarle al asistente.
✓ **Legal, cuando corresponde**	La responsabilidad no se transfiere a la herramienta — independientemente de quién redactó materialmente el código.

Documentación técnica mínima del código

No se trata de documentar todo, sino de documentar lo que no es evidente en el código mismo.

✓ **Contratos de interfaces**	Precondiciones y postcondiciones de cada función pública.
✓ **Decisiones no obvias**	Por qué se eligió este algoritmo frente a otros disponibles.
✓ **Supuestos registrados**	Qué condiciones del entorno o de los datos podrían invalidar la solución si cambian.

Síntesis de la semana 3

Diseño y estándares

- ✓ Encapsular los puntos de variación con el mecanismo adecuado.
- ✓ Reutilizar antes de duplicar.
- ✓ Dejar que el formateador decida el estilo; el equipo decide la nomenclatura.
- ✓ Comentar el porqué, nunca el qué.

Construcción con IA

- ✓ Especificar el prompt como se especifica una función.
- ✓ Descomponer antes de generar.
- ✓ Revisar todo código generado con el estándar de la semana 6.
- ✓ Asumir la autoría — y la responsabilidad — del código integrado.

REFERENCIAS

Bibliografía

- ✓ McConnell, S. (2004). Code Complete: A Practical Handbook of Software Construction (2.ª ed.). Microsoft Press.
- ✓ Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley. (Origen del patrón Strategy.)
- ✓ Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2.ª ed.). Addison-Wesley.
- ✓ Denny, P., Kumar, V., Giacaman, N. et al. (2024). Investigaciones sobre estrategias de prompting y descomposición de tareas en la generación de código asistida por IA.
- ✓ IEEE Computer Society (2024). Guide to the Software Engineering Body of Knowledge (SWEBOK), Version 4.0.
- ✓ Prettier, Black (Python) y gofmt (Go) — documentación oficial de formateadores automáticos de código.